



Development Tasks & Enhancement Roadmap

Executive Summary

SentinelAI is an AI-powered Security Operations Center (SOC) platform designed to automate threat detection, accelerate incident response, and provide actionable intelligence to security analysts. This document outlines the planned development tasks, enhancements, and engineering priorities for the current release cycle.

The platform integrates machine learning-based anomaly detection, a real-time AI SOC Copilot, MITRE ATT&CK framework mapping, and automated response workflows. The roadmap presented here covers five core engineering domains:

#	Domain	Focus Area
1	Security & Access Management	JWT authentication, RBAC, session security
2	ML Model Optimization	Threat detection accuracy, false positive reduction
3	AI SOC Copilot Enhancement	Alert analysis, attack explanations, remediation
4	Threat Intelligence & MITRE ATT&CK	VirusTotal, AbuseIPDB, OTX, ATT&CK mapping
5	Dashboard & Automated Response	Real-time analytics, escalation workflows

Current Sprint Priorities

The following five items are actively in development during the current sprint cycle:

- 1. ML model optimization and detection accuracy improvement
- 2. AI SOC Copilot enhancements — alert summarization and investigation workflows
- 3. Threat intelligence and MITRE ATT&CK mapping improvements
- 4. JWT authentication and Role-Based Access Control (RBAC) implementation
- 5. Dashboard performance and automated response workflow optimization

1. Security & Access Management

Robust authentication and authorization are foundational to the SentinelAI platform. This section covers the implementation of industry-standard security mechanisms to protect platform access and enforce least-privilege principles.

1.1 JWT Authentication

Implement JSON Web Token (JWT) based authentication across all API endpoints. Tokens will be short-lived with refresh token rotation, signed using RS256 asymmetric keys, and validated at the API gateway layer.

- Issue signed JWTs on login with 15-minute expiry
- Implement refresh token rotation with 7-day sliding window
- Validate tokens at API gateway — reject expired or tampered tokens
- Store refresh tokens securely in HTTP-only cookies

1.2 Role-Based Access Control (RBAC)

Define and enforce granular permission models across all platform modules. Roles will be hierarchical with scope-based access to dashboards, reports, threat data, and configuration panels.

- Define roles: Admin, SOC Analyst, Read-Only Viewer, Incident Responder
- Map permissions per module (Dashboard, Reports, Config, Incident Response)
- Enforce RBAC checks at controller and service layers
- Build role assignment and audit UI in the Admin panel

1.3 Secure Session Management

- Implement idle session timeout (configurable, default 30 min)
- Force re-authentication on sensitive actions (e.g. role changes, exports)
- Log all authentication events to the SOC audit trail
- Support MFA via TOTP for admin-tier accounts

2. ML Model Optimization

The threat detection engine relies on ML models trained to identify brute force, DoS/DDoS, lateral movement, and data exfiltration patterns. This section focuses on improving model accuracy, reducing noise, and enhancing the confidence scoring pipeline.

2.1 Fine-Tuning & Accuracy

- Retrain models on updated 2025–2026 threat datasets
- Add cross-validation across network segment types
- Introduce ensemble methods (XGBoost + LSTM) for hybrid detection
- Achieve target F1-score ≥ 0.94 across all threat categories

2.2 False Positive Reduction

- Implement dynamic threshold tuning per asset profile
- Add analyst feedback loop — thumbs up/down on alerts feeds retraining
- Deploy A/B testing for model versions in staging before rollout
- Target false positive rate $\leq 3\%$ in production

2.3 Confidence Scoring

- Normalize confidence scores to 0–100 scale across all models
- Surface confidence alongside each alert in the SOC dashboard
- Auto-suppress alerts below configurable confidence floor (default: 40)
- Include feature importance breakdown in alert detail view

3. AI SOC Copilot Enhancement

The AI SOC Copilot serves as an intelligent assistant to security analysts — summarizing alerts, explaining attack chains, recommending remediation steps, and guiding investigation workflows. Enhancements focus on depth, accuracy, and analyst experience.

3.1 Alert Analysis & Summarization

- Generate natural-language summaries for every triggered alert
- Include asset context, timeline, and contributing indicators
- Provide severity justification with supporting evidence
- Summarize multi-stage attack chains into concise narratives

3.2 Attack Explanation & Remediation

- Map detected behaviors to MITRE ATT&CK tactics and techniques
- Generate step-by-step remediation playbooks per alert type
- Link recommended actions to CVE references where applicable
- Offer rollback guidance for failed/rolled-back patch deployments

3.3 Investigation Workflow Improvements

- Add 'Ask Copilot' contextual chat within each alert detail panel
- Enable Copilot to query historical incidents for pattern comparison
- Support analyst annotations that inform future Copilot responses
- Integrate Copilot suggestions into incident response ticket creation

4. Threat Intelligence & MITRE ATT&CK

Threat intelligence integrations provide enriched context for every detected event. This section covers improvements to external feed integrations, correlation logic, and ATT&CK framework coverage.

4.1 External Feed Integrations

Feed	Status	Enhancement
VirusTotal	Active	Add hash + URL reputation lookup
AbuseIPDB	Active	Increase confidence weight for known bad IPs
AlienVault OTX	Active	Subscribe to threat pulse feeds by sector
CISA KEV	Planned	Pull known exploited vulnerability list daily

4.2 Threat Correlation & ATT&CK Mapping

- Improve multi-event correlation windows from 5 min → 30 min
- Map 100% of alert types to ATT&CK Tactic + Technique pairs
- Build heatmap view of ATT&CK matrix showing coverage gaps
- Auto-generate ATT&CK Navigator layer from detected techniques
- Correlate related alerts into unified incident records

5. Dashboard & Automated Response Optimization

The SOC Dashboard is the primary interface for analysts. This section covers performance improvements, enhanced analytics, and intelligent automation for response workflows.

5.1 Real-Time Dashboard Performance

- Migrate alert feed to WebSocket for sub-second push updates
- Implement virtual scrolling for large alert lists (10,000+ events)
- Add widget-level caching with 5-second TTL for metric tiles
- Optimize backend aggregation queries — target P95 < 200ms

5.2 Risk Scoring & Analytics

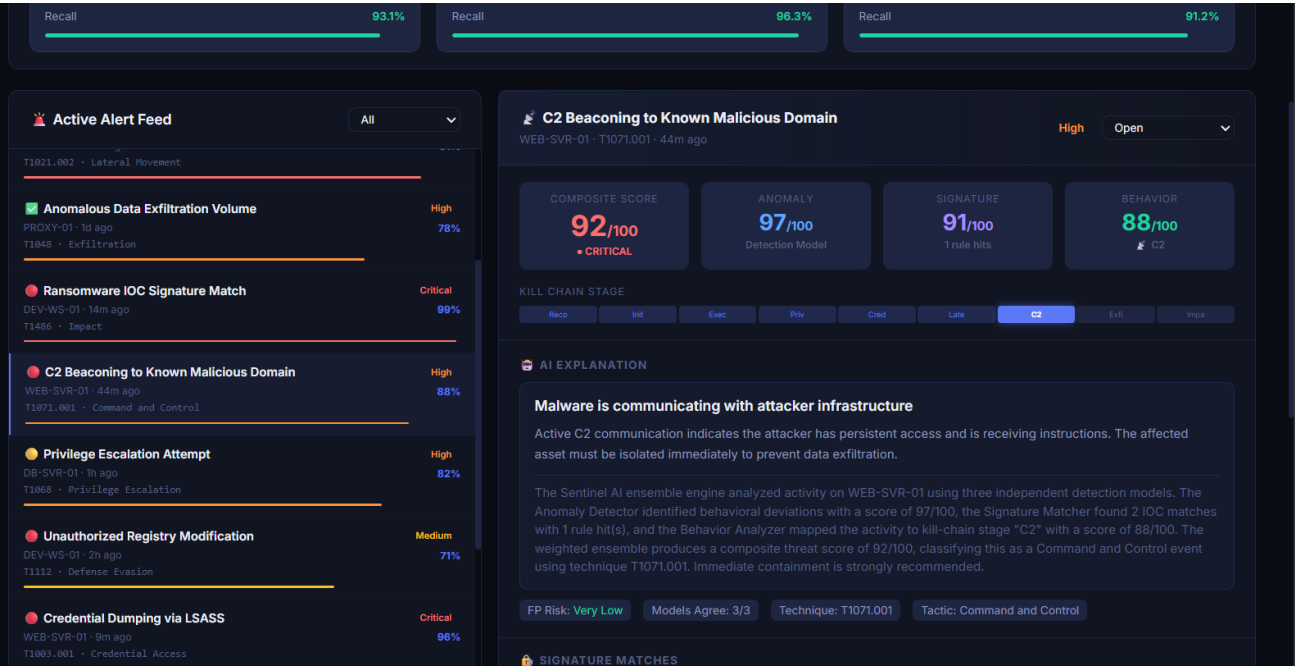
- Introduce dynamic asset risk scoring based on patch posture + threat level
- Add trend charts: 7-day, 30-day, 90-day alert volumes by severity
- Build analyst performance metrics: MTTD, MTTR per analyst
- Export analytics to PDF/CSV from the Reports module

5.3 Automated Response Workflows

- Build no-code playbook editor for common response scenarios
- Auto-isolate assets when Critical threat confidence > 85%
- Trigger patch assignment from alert — one-click from Copilot suggestion
- Send Slack/email escalation when SLA thresholds are breached

Delivery Timeline

Phase	Focus Area	Target Date	Status
Phase 1	JWT Auth & RBAC	July 2026	In Progress
Phase 2	ML Model Retraining	August 2026	Planned
Phase 3	SOC Copilot v2	September 2026	Planned
Phase 4	Threat Intel & ATT&CK	October 2026	Planned
Phase 5	Dashboard Optimization	November 2026	Planned



Printer

Save as PDF

Layout

☒ Portrait
 ☐ Landscape

Pages

☒ All
 ☐ Odd pages only
 ☐ Even pages only

More settings

Print using system dialog... (Ctrl+Shift+P)

Save Cancel

26:56 Logout

Asset	Asset Type	Missing Patch	Severity
WEB-SVR-01	Server	OpenSSL Security Update	High
DB-SVR-01	Server	OpenSSL Security Update	High
DEV-WS-01	Workstation	OpenSSL Security Update	High
DEV-WS-02	Workstation	OpenSSL Security Update	High
PROXY-01	Network	OpenSSL Security Update	High
STORAGE-	Server	OpenSSL Security	High

Code-

```

let currentPage = 'dashboard';
let currentPatchPage = 'dashboard';

function showPage(page) {
  currentPage = page;
  // Refresh auth session on activity
  if (typeof auth !== 'undefined' && auth.currentUser) auth._startSessionTimer();

  document.querySelectorAll('.nav-item').forEach(el => el.classList.remove('active'));
  const pageLabels = {
    dashboard:'SOC Dashboard', patch:'Patch Management',
    copilot:'AI Copilot', threat:'Threat Intelligence',
    mitre:'MITRE ATT&CK', reports:'Reports',
  };
  document.querySelectorAll('.nav-item').forEach(el => {
    if (el.textContent.trim().includes(pageLabels[page] || page))
      el.classList.add('active');
  });

  if (page === 'patch') renderPatchModule('dashboard');
  else if (page === 'reports') renderPatchModule('reports');
  else if (page === 'copilot') _renderPage(renderCopilot());
  else if (page === 'threat') { renderThreatModule(); }
  else if (page === 'mitre') _renderPage(renderMitreAttack());
}

```

```

    else renderSOCDashboard();
  }

function _renderPage(html) {
  document.getElementById('mainContent').innerHTML = html;
}

function showPatchPage(sub) { currentPatchPage = sub; renderPatchModule(sub); }
function refreshPatchSubPage(sub) { renderPatchModule(sub || currentPatchPage); }

function renderPatchModule(subPage) {
  currentPatchPage = subPage;
  const main = document.getElementById('mainContent');
  const tabs = [
    {id:'dashboard', label:' Dashboard'},
    {id:'repo', label:' Repository'},
    {id:'assign', label:' Assignment'},
    {id:'monitor', label:' Monitoring'},
    {id:'reports', label:' Reports'},
  ];
  const subNav = `<div style="background:var(--bg-card);border-bottom:1px solid var(--border);padding:0 32px;display:flex;gap:4px;">
    ${tabs.map(t=>`<button onclick="showPatchPage('${t.id}')"
      style="padding:10px 16px;background:none;border:none;border-bottom:2px solid
    ${subPage===t.id?'var(--accent)':'transparent'};
      color:${subPage===t.id?'var(--accent)':'var(--text-2)}';cursor:pointer;font-family:inherit;font-size:13px;
      font-weight:${subPage===t.id?600:400};transition:all .18s;">${t.label}</button>`).join("")}
  </div>`;
  let content = "";
  if (subPage==='dashboard') content = renderPatchDashboard();
  else if (subPage==='repo') content = renderPatchRepo();
  else if (subPage==='assign') content = renderPatchAssignment();
  else if (subPage==='monitor') content = renderMonitor();
  else if (subPage==='reports') content = renderReports();
  main.innerHTML = subNav + content;
  if (subPage==='dashboard')
  requestAnimationFrame(()=>requestAnimationFrame(initDashboardCharts));
}

function renderSOCDashboard() {
  const main = document.getElementById('mainContent');
  const s = db.stats();
  const as = db.alertStats();
  const risk = db.overallRisk();
  const alerts = db.getAlerts().filter(a => a.status !== 'Resolved').slice(0, 8);
  const assets = db.getAssets();
  const sevColor = { Critical:'var(--red)', High:'var(--orange)', Medium:'var(--yellow)', Low:'var(--green)'};
  const statIcon = { Open:', Investigating:', Resolved:'};

  main.innerHTML = `
<div class="page-header fade-in">
  <div>

```

```

    <div class="page-title"> SOC Overview Dashboard</div>
    <div class="page-sub">Real-time security posture — Sentinel AI v2.0</div>
  </div>
  <div style="display:flex;align-items:center;gap:12px;">
    <div style="display:flex;align-items:center;gap:6px;font-size:12px;color:var(--green);">
      <span class="dot dot-green pulse"></span>All Systems Operational
    </div>
    <button class="btn btn-ghost btn-sm" onclick="renderSOCDashboard()"> Refresh</button>
    ${auth.canDo('automated_response')} ? `<button class="btn btn-primary btn-sm"
  onclick="showPage('copilot');generateNewAlert()"> Inject Alert</button>` : ""
  </div>
</div>

<div class="page-body fade-in">

  <!-- Top Stats Row -->
  <div class="stats-grid" style="grid-template-columns:repeat(6,1fr);">
    <div class="stat-card red">
      <div class="stat-label">Critical Alerts</div>
      <div class="stat-value red">${as.critical}</div>
      <div class="stat-sub">Open & unresolved</div>
    </div>
    <div class="stat-card orange">
      <div class="stat-label">High Alerts</div>
      <div class="stat-value" style="color:var(--orange);">${as.high}</div>
      <div class="stat-sub">Open & unresolved</div>
    </div>
    <div class="stat-card yellow">
      <div class="stat-label">Investigating</div>
      <div class="stat-value yellow">${as.invest}</div>
      <div class="stat-sub">In progress</div>
    </div>
    <div class="stat-card green">
      <div class="stat-label">Compliance Rate</div>
      <div class="stat-value green">${s.compliance}%</div>
      <div class="stat-sub">Patch compliance</div>
    </div>
    <div class="stat-card blue">
      <div class="stat-label">Patches Pending</div>
      <div class="stat-value blue">${s.pending}</div>
      <div class="stat-sub">Awaiting deploy</div>
    </div>
    <div class="stat-card purple">
      <div class="stat-label">ML Confidence</div>
      <div class="stat-value" style="color:var(--accent2);">${as.avgConf}%</div>
      <div class="stat-sub">Avg detection conf.</div>
    </div>
  </div>

  <!-- Row 2: Risk Gauge + Alert Feed + Asset Health -->
  <div style="display:grid;grid-template-columns:200px 1fr 300px;gap:16px;margin-bottom:20px;">

    <!-- Risk Gauge -->
    <div class="panel">

```



```

<div class="panel-header"><div class="panel-title"> Risk Score</div></div>
<div class="panel-body" style="text-align:center;padding:20px 16px;">
  <div style="position:relative;display:inline-flex;align-items:center;justify-content:center;margin-
bottom:12px;">
    <canvas id="riskGaugeCanvas" width="140" height="140"></canvas>
    <div style="position:absolute;text-align:center;">
      <div style="font-size:26px;font-weight:800;color:${risk.color};">${risk.score}</div>
      <div style="font-size:9px;color:var(--text-3);text-transform:uppercase;letter-
spacing:.06em;">/ 100</div>
    </div>
  </div>
  <div style="font-size:14px;font-weight:700;color:${risk.color};">● ${risk.level} Risk</div>
  <div style="font-size:10px;color:var(--text-3);margin-top:4px;">Organization-wide</div>
  <div style="margin-top:14px;display:flex;flex-direction:column;gap:6px;">
    ${assets.map(a => {
      const r = db.riskScore(a.id);
      return `<div style="display:flex;align-items:center;gap:6px;font-size:10px;">
        <span style="color:${r.color};">●</span>
        <span style="color:var(--text-2);flex:1;overflow:hidden;text-overflow:ellipsis;white-
space:nowrap;">${a.name}</span>
        <span style="color:${r.color};font-weight:600;">${r.score}</span>
      </div>`;
    }).join("")}
  </div>
</div>
</div>

<!-- Live Alert Feed -->
<div class="panel">
  <div class="panel-header">
    <div class="panel-title"> Live Threat Feed</div>
    <div style="display:flex;gap:6px;align-items:center;">
      <span class="dot dot-red pulse"></span>
      <span style="font-size:11px;color:var(--text-3);" id="feedLastUpdate">Live</span>
      <button class="btn btn-ghost btn-sm" onclick="showPage('copilot')>View All →</button>
    </div>
  </div>
  <div style="max-height:320px;overflow-y:auto;" id="dashAlertFeed">
    ${alerts.map(al => `
      <div class="dash-alert-item"
onclick="showPage('copilot');setTimeout(()=>selectCopilotAlert('${al.id}'),100)">
        <div style="display:flex;align-items:center;gap:8px;padding:10px 16px;border-bottom:1px
solid rgba(99,120,220,.06);cursor:pointer;transition:background .15s;">
          <span style="font-size:16px;">${al.severity==='Critical'?":al.severity==='High'?":""}</span>
          <div style="flex:1;min-width:0;">
            <div style="font-size:12px;font-weight:600;color:var(--text-1);white-
space:nowrap;overflow:hidden;text-overflow:ellipsis;">${al.title}</div>
            <div style="font-size:10px;color:var(--text-3);">${al.asset} · ${timeAgo(al.timestamp)} ·
${al.technique}</div>
          </div>
          <div style="flex-shrink:0;text-align:right;">
            <span class="badge"
style="background:${sevColor[al.severity]}22;color:${sevColor[al.severity]};font-
size:9px;">${al.severity}</span>

```

```

    <div style="font-size:10px;color:var(--accent);font-weight:600;margin-
top:2px;">${al.confidence}%</div>
    </div>
  </div>
  <div>`).join("") || '<div style="padding:40px;text-align:center;color:var(--text-3);">No open alerts
</div>}'
</div>
</div>

<!-- Asset Health Map -->
<div class="panel">
  <div class="panel-header"><div class="panel-title"> Asset Health Map</div></div>
  <div class="panel-body" style="padding:14px;">
    <div style="display:grid;grid-template-columns:1fr 1fr;gap:8px;">
      ${assets.map(a => {
        const r = db.riskScore(a.id);
        const openAlerts = db.getAlerts().filter(al => al.assetId === a.id && al.status !==
'Resolved').length;
        return `
          <div class="asset-health-card" style="border-color:${r.color}33;cursor:pointer;"
onclick="showPage('patch');showPatchPage('monitor')" title="${a.name} — ${r.level} Risk">
            <div style="display:flex;align-items:center;gap:6px;margin-bottom:4px;">
              <span style="color:${r.color};font-
size:16px;">${a.type==='Server'?a.type==='Workstation'?}</span>
              <div>
                <div style="font-size:10px;font-weight:700;color:var(--text-1);">${a.name}</div>
                <div style="font-size:9px;color:var(--text-3);">${a.type}</div>
              </div>
            </div>
            <div style="display:flex;justify-content:space-between;align-items:center;">
              <div style="font-size:16px;font-weight:800;color:${r.color};">${r.score}</div>
              ${openAlerts > 0 ? `<span class="badge"
style="background:${r.color}22;color:${r.color};font-size:9px;">${openAlerts} alerts</span>` : `<span
style="font-size:9px;color:var(--green);"> Clean</span>`}
            </div>
            <div class="progress-bar" style="height:3px;margin-top:6px;">
              <div class="progress-fill" style="width:${r.score}%;background:${r.color};"></div>
            </div>
          </div>`;
      }).join("")}
    </div>
  </div>
</div>

<!-- Row 3: Automated Response + MITRE Quick + Patch Summary -->
<div style="display:grid;grid-template-columns:1fr 1fr 1fr;gap:16px;">

  <!-- Automated Response Playbooks -->
  <div class="panel">
    <div class="panel-header">
      <div class="panel-title"> Response Playbooks</div>
      <span class="tag">${db.getPlaybooks().filter(pb=>pb.enabled).length} active</span>
    </div>
  </div>

```

```
<div class="panel-body" style="padding:10px 14px;">
  ${db.getPlaybooks().map(pb => `
    <div style="display:flex;align-items:center;gap:8px;padding:8px 0;border-bottom:1px solid
rgba(99,120,220,.06);">
      <div class="playbook-status-dot" style="background:${pb.enabled?'var(--green)':'var(--text-
3)}';"></div>
      <div style="flex:1;min-width:0;">
        <div style="font-size:11px;font-weight:600;color:var(--text-1);white-
space:nowrap;overflow:hidden;text-overflow:ellipsis;">${pb.name}</div>
        <div style="font-size:9px;color:var(--text-3);">${pb.actions.slice(0,2).join(',
')}${pb.actions.length>2?'...':""}</div>
      </div>
      ${auth.canDo('automated_response')} ? `
      <label class="toggle-switch" title="${pb.enabled?'Disable':'Enable'} playbook">
        <input type="checkbox" ${pb.enabled?'checked':''}
onchange="db.togglePlaybook('${pb.id}');renderSOCDashboard()"/>
        <span class="toggle-slider"></span>
      </label>` : `<span style="font-size:9px;color:${pb.enabled?'var(--green)':'var(--text-
3)}';">${pb.enabled?'ON':'OFF'}</span>`
    </div>`).join("")}
  </div>
</div>
```